

AUTOMATA LEARNING LAB · O'REILLY LIVE TRAINING

Automate Tasks with *Python + AI*

Read it. Run it. Steer it. — You don't need to write Python.

You need to read the Python an AI writes *for* you.

 LIVE DEMO

COLD OPEN

Let's automate something. *Right now.*

```
uv run demos/reserve/downloads-organizer/organize.py ~/mess-demo  
uv run demos/reserve/downloads-organizer/organize.py ~/mess-demo --apply
```

A folder with 40 messy files → sorted into tidy subfolders. Live.

What just happened?



Every automation in this course — every automation, period — is this shape.

POLL 1

What would *you* automate?

1. Files & folders (sorting, renaming, backups)
2. Documents (PDFs, invoices, reports)
3. Data (spreadsheets, dashboards)
4. Communication (email, summaries, briefings)
5. Something weirder — tell us in chat

PART 0

Why automate — and *why now*

The math of small chores

15 min

a day of file-sorting, copy-pasting, renaming, reformatting...

65 hrs

that's a year of it — a month and a half of workdays

30 min

what a script that erases the chore forever now costs to build

- The chores are invisible because they're small. **They compound; so do the scripts**
- An automation is the only work you do once and get paid for daily

Why Python?

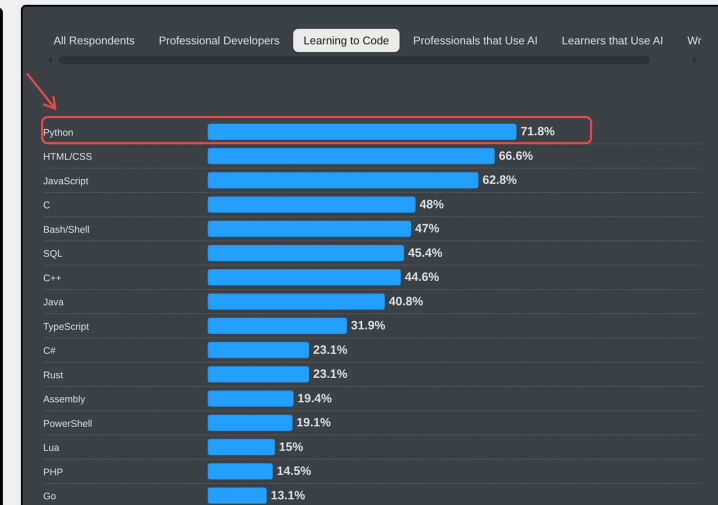
- The **most popular** programming language on Earth — for a decade running
- The **glue of the computer**: files, spreadsheets, PDFs, websites, email — there's a package for everything
- And the kicker for this course: it's the language **AI** **write best** — more Python in their training data than anything else

Most popular technologies 2.1

Programming, scripting, and markup languages

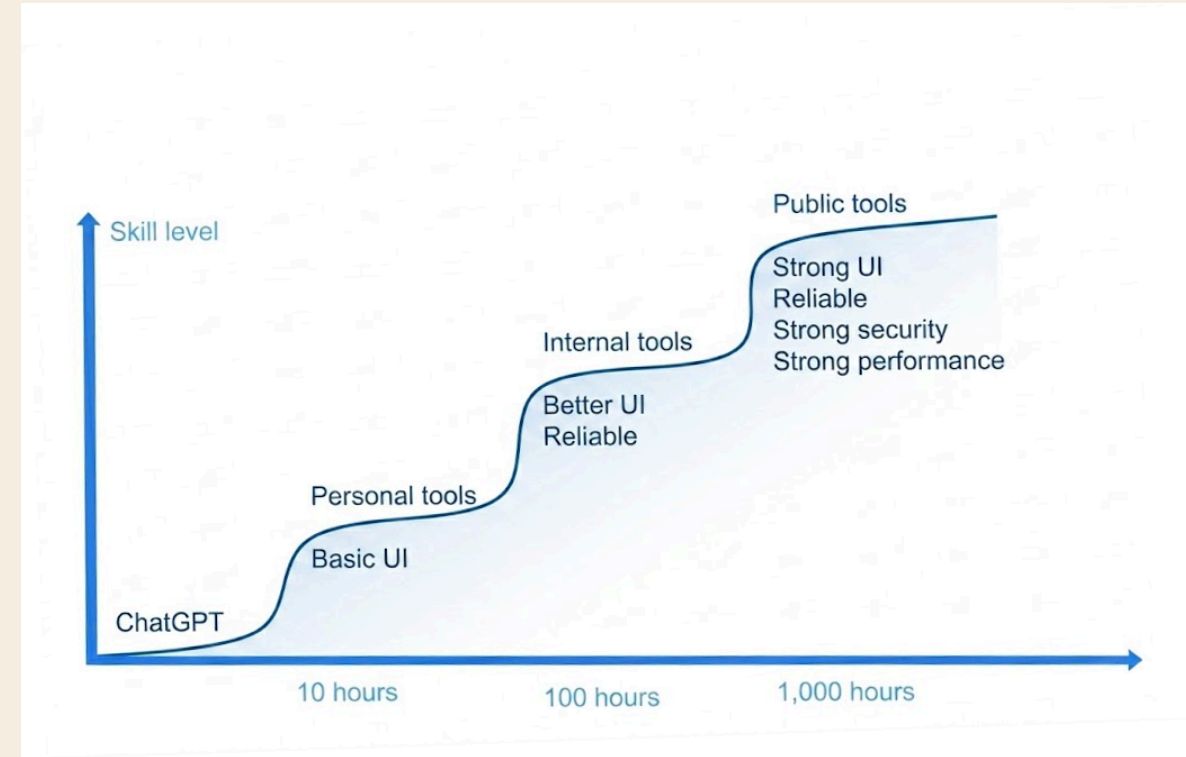
After more than a decade of steady growth, Python's adoption has accelerated significantly. It saw a 7 percentage point increase from 2024 to 2025; this speaks to its ability to be the go-to language for AI, data science, and back-end development.

2 Which programming, scripting, and markup languages have you done extensive development work in over the past year, and which do you want to work in over the next year? (If you both worked with the language and want to continue to do so, please check both boxes in that row.)



Why now?

- This curve used to start with a **6-month syntax climb** before your first useful tool
- AI collapsed the wall between *having an idea* and *having a tool*
- **Personal tools now sit ~10 hours in** — this course *is* that first step
- You're not late to programming. You're **exactly on time** for this version of it



Programming, demystified

- A program is a **recipe written for a very literal cook**
- The cook (your computer) does *exactly* what the recipe says — brilliantly, instantly, and with zero common sense
- Old world: you had to write the recipe yourself, in the cook's strange language
- New world: **an AI drafts the recipe — your job is to check it before the cook starts cooking**
- That checking skill is what the next two days build

"Vibe coding" — and why we're not doing it

- *"Fully give in to the vibes... forget that the code even exists"* — now a dictionary word
- It works — right up until the script touches **your real files**
- You can debug a wrong output. You **can't un-delete** a folder
- This course: *vibe coding, but you check the mirrors first*

The third lane

	Coding bootcamp	Vibe - coding course	This course
Goal	Become a developer	Ship a toy app	Supervise AI that codes
Python	Years of syntax	"Forget the code exists"	A reading vocabulary
Safety	Assumed	Ignored	The core skill
Ends at	A job hunt	A deployed demo	Scheduled, unattended tools

What you leave with

The SCRIPT loop

One repeatable formula for safely generating and running automations — the course on a page.

A reading vocabulary

~20 Python concepts — enough to inspect any script.

6 working tools

Built live, yours to keep and modify.

Safety habits

Run Gate, dry-runs, done-means checks.

Take-home prompts

The ticket template + generation prompt + checklists + a build-an-automation-for-MY-machine prompt (Mac & Windows), in `prompts/`.

AGENDA

The two days

1. **Day 1 — Read Python, run scripts.** The 5-minute setup, then the reading vocabulary: variables → loops → decisions → files → packages → APIs → AI. Capstone: an organizer you can read every line of.
2. **Day 2 — The loop at full power.** Three real automations, one formula, real time to modify each — ending with a script that runs **unattended, on a schedule**, then upgraded into an **agent-callable skill**.

Six words you'll hear all week

Word	What it means
terminal	The text window where you type commands. That's the whole app.
command	One instruction you type + Enter. <code>uv run script.py</code> is a command.
flag	An option after a command: <code>--help</code> explains, <code>--apply</code> makes it real.
dry run	The script only <i>prints</i> what it would do. Rehearsal, not performance.
alias	A one - word nickname you give a long command. "Shipping" a tool = aliasing it.
schedule	Your computer running a command by itself, on a timer, without you.

Nothing else is assumed. Anything unfamiliar past this slide is a bug — call it out.

PART 1

The formula: the *SCRIPT* loop

The SCRIPT loop

S — Spot it

Worth automating? Does it already exist?

C — Compose the ticket

One-paragraph spec.

R — Request the code

One prompt → one file.

I — Inspect

The Run Gate. Read before you run.

P — Prove it

Dry-run → verify "done".

T — Turn into a tool

Alias it. Schedule it.

S — Spot it: which zone is your task in?

-  **Green** — repetitive, rule-based, checkable: files, formats, fetches, reports → *automate today*
-  **Yellow** — needs judgment an AI can add: classify, summarize, extract → *automate with AI in the middle*
-  **Red** — irreversible, high-stakes, or you can't define "done" → *keep a human on it*

The intuition you're building all week: **sorting your tasks into these zones.**

S — Spot it: ask before you build

- "Does a tool for this **already exist**?" — built-in OS feature, free app, no-code service
- Two minutes of asking beats two hours of building — prompt in `prompts/does-this-exist.md`
- Honest answer about no-code (Zapier, n8n): great guardrails, **can't touch your local files** — that gap is exactly where Python scripts live

C — Compose the ticket

```
TASK:      what should happen, one sentence
TRIGGER:    manual / every morning / when a file appears
TOUCHES:    exact folders, files, sites it may read or write
MUST NEVER: delete, send, leave folder X, spend money...
DONE MEANS: the concrete result you will check by hand
```

- A vague ask produces a vague — and riskier — script
- If you can't fill in **DONE MEANS**, you can't verify it. Don't build it yet

R — Request the code



- That was the 2025 workflow — it works. **SCRIPT wraps it in seatbelts**
- Demand: **single file · uv inline deps · dry-run by default · no classes · friendly errors**
- Show the AI **how to call the API** — don't let it guess. One task, one file. Disposable is fine

I — Inspect: the Run Gate

Ask the script...	If yes →
Do you write / move / delete files?	dry-run first, practice folder
Do you touch the network ?	know which sites, what data leaves
Do you read my keys ?	keys live in <code>.env</code> , never in code
Do you install / run other software?	understand exactly what — or stop
Contain <code>subprocess</code> / <code>os.system</code> / <code>eval</code> ?	red flags — stop, ask what gets executed
Will you run unattended ?	all of the above, stricter

The package check

~1 in 5

AI-suggested packages don't exist

pypi.org

30-second habit: look up every unfamiliar name

0

typos that are "close enough" — squatters register the fakes

P — Prove it

- **Dry-run first.** Read what it *would* do — every line
- `--apply` on a **practice copy**, then the real thing
- Check the ticket's **DONE MEANS** by hand — Red → Green
- You told the AI what done means. *Now hold it to that.*

T — Turn it into a (reusable) tool

The script is cheap. The reusable **capability** is the asset. Finish it so a human *or* an agent could use it: **clear inputs · clear outputs · safe defaults · a usage contract**. Then ship it up the ladder:

- **Alias it** — a one-word nickname: `alias tidy='uv run ~/tools/organize.py ~/Downloads'`
- **Schedule it** — your machine runs it on a timer: launchd / Task Scheduler — Day 2, automation 3
- **Skill it** (optional) — one markdown file so an agent can invoke it — Day 2 endpoint



PART 2 · DAY 1

Just enough Python: a *reading vocabulary*

Not a syllabus. ~20 concepts — the ones you need to audit any script an AI hands you.



FOLLOW ALONG — EVERYONE

Setup: the whole thing

1

command to install (uv — Mac & Windows)

```
uv run lessons/01_first_run.py
```

0

Python installs, venvs, PATH edits, kernel
pickers

5 min

to your first running script

— uv downloads Python itself, installs each script's deps, runs it.

👉 FOLLOW ALONG — EVERYONE

Terminal survival kit (all five of them)

- **Open it** — Mac: ⌘-space → "Terminal" · Windows: Start → "PowerShell"
- `cd foldername` — go into a folder (`cd ..` = go back up)
- `ls` (Mac) / `dir` (Windows) — what's in here?
- **↑ arrow** — repeat the last command (you'll use this constantly)
- **Tab** — auto-complete file names. Never type a long path again
- Bonus: **Ctrl-C** — stop a running script. You can't break anything today

Why this setup works (and the old one didn't)

The old way	With uv
Install Python (which one?)	—
Create a venv, activate it	—
<code>pip install -r requirements.txt</code>	—
"python vs python3"? PATH?	—
Finally run something	<code>uv run script.py</code>

Every script declares its own needs in its header. That's the whole story.

 LIVE DEMO

LESSON 01 · VARIABLES & F-STRINGS

```
name = "Ada"  
downloads = 47  
  
# — PREDICT: what will this print? —  
print(f"Hi {name}, you have {downloads} files to sort.")
```

- A **variable** is a labeled box. An **f-string** fills blanks into text
- Why you care: `f"moving {file} → {folder}"` is how scripts *tell you what they're doing*

Prediction-first: how we'll read code all week

- Before every run: **"What will this print?"** — commit to a guess
- Right guess → the concept is yours. Wrong guess → *that's the lesson*
- This is the exact skill of the Run Gate: predicting what a script does **before** it does it

 LIVE DEMO

LESSON 02 · LISTS & LOOPS

```
files = ["report.pdf", "photo.jpg", "notes.txt", "invoice.pdf"]

# — PREDICT: how many lines print? —
for f in files:
    if f.endswith(".pdf"):
        print(f"PDF found: {f}")
```

- A **list** holds many things; a **loop** does something to each
- Every automation ever: *"for each file... for each row... for each email..."*

 LIVE DEMO

LESSON 03 · DECISIONS & FUNCTIONS

```
def folder_for(filename):  
    if filename.endswith((".jpg", ".png")):  
        return "Images"  
    elif filename.endswith(".pdf"):  
        return "Documents"  
    else:  
        return "Everything else"
```

- **if / elif / else** = the rules; a **function** = a named, reusable rule
- Reading AI code is mostly reading *its* rules and asking: **are these my rules?**

POLL 2 · PREDICT

folder_for("taxes-2026.pdf") returns...

1. "Images"
2. "Documents"
3. "Everything else"
4. an error

👉 FOLLOW ALONG — EVERYONE

Puzzle break: rebuild the script

```
uv run lessons/puzzles.py
```

- The lines of a small script, **shuffled** — you put them back in order
- Reordering forces you to see the *skeleton*: setup → loop → decision → output
- Cheaper than writing, just as sticky — and it's the same skill as scanning AI code

 LIVE DEMO

LESSON 04 · FILES & PATHS

```
from pathlib import Path

downloads = Path.home() / "Downloads"
# — PREDICT: does this CHANGE anything on disk? —
for item in downloads.iterdir():
    print(item.name, "→", folder_for(item.name))
```

- `Path` is how Python points at your real files — **reading is safe, writing is the Run Gate's business**
- Spot the difference: `print(...)` vs `shutil.move(...)` — *that's* the line you audit

 LIVE DEMO

LESSON 05 · PACKAGES & DEPENDENCIES

```
# /// script
# requires-python = "≥3.12"
# dependencies = ["requests", "rich"]
# ///
import requests
```

- A **package** is code someone else wrote; a **dependency** is a package your script needs
- This header (PEP 723) makes a script **self-contained**: `uv run` reads it and sets everything up
- Run Gate tie-in: this list is exactly where you do the **pypi.org check**

 LIVE DEMO

LESSON 06 · WHEN IT BREAKS

```
Traceback (most recent call last):  
  File "briefing.py", line 42, in <module>  
    print(story["headline"])  
KeyError: 'headline'
```

- Read the **last line first**: the error type + what was missing
- Then find the line number **in your file** — skip the library noise
- `try/except` around risky lines = the script's **safety net**; when auditing AI code, notice when the net is *missing*
-  Red flags to stop on: `subprocess.run` · `os.system` · `eval()` — "what exactly does this execute?"

 LIVE DEMO

LESSON 07 · TALKING TO APIS

```
import requests

url = "https://api.open-meteo.com/v1/forecast?latitude=38.7&longitude=-9.1&current_weather=true"
weather = requests.get(url).json()
# — PREDICT: what type is `weather`? —
print(weather["current_weather"]["temperature"])
```

- An **API** = a website for programs: you ask a URL, you get structured data back
- Dictionaries (`{ ... }`) are how that data arrives — keys in, values out

 LIVE DEMO

LESSON 08 · TALKING TO AI — IT'S JUST ANOTHER API

```
from anthropic import Anthropic

client = Anthropic() # key comes from .env — never from the code
response = client.messages.create(
    model="claude-opus-4-8", max_tokens=16000,
    messages=[{"role": "user", "content": "Summarize this receipt: ..."}])
```

- Same shape with OpenAI, same shape with a **local** model (Ollama) — swap one line
- The AI call is just one more **PROCESS** step between INPUT and OUTPUT

Keys: the three rules

- Keys live in `.env` — a file on your machine, never inside scripts
- Never paste a key into a chat, a script, or a screenshot
- If a script wants your keys, that's a **Run Gate question**: *which* key, sent *where*?

 LIVE DEMO

Day 1 capstone: read every line

- `lessons/09_capstone_organizer.py` — the cold-open organizer, ~160 lines
- Variables ✓ loops ✓ decisions ✓ functions ✓ paths ✓ dry-run flag ✓
- **This morning it was magic. Now it's just Python you can read.**

What you can now read

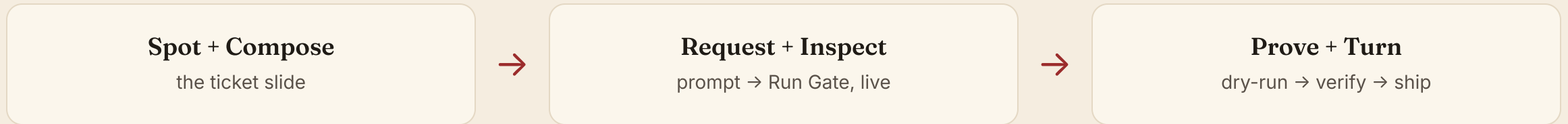
- variables · f-strings · lists · loops · dictionaries · indexing & slicing
- if / elif / else · functions · return · `.strip()` `.split()` `.join()`
- `pathlib` paths · read vs **write** operations · `with open(...)` (recognize)
- imports · the PEP 723 dependency header · comprehensions (recognize)
- **a traceback** · try/except ·  `subprocess` / `eval` red flags
- `requests` + JSON · an AI API call · `.env` keys · `--help` / `--apply` flags

That's the vocabulary. Tomorrow: six full SCRIPT reps.

PART 3 · DAY 2

Three automations, *one formula*

Every demo today, same beats



Three reps, not six — so we have time to **modify, inspect, and code-walk** each one. The automations get more impressive. **The formula never changes.**

...and after every demo: 🖋️ **your turn** — one 5-minute modification.

The tool isn't yours until you've changed it.

The ticket

TASK: Triage a folder of MIXED documents (invoices, notes, receipts, screenshots) into one table: type · summary · priority · dates · people · action items.

TRIGGER: Manual — I run it against a folder I name.

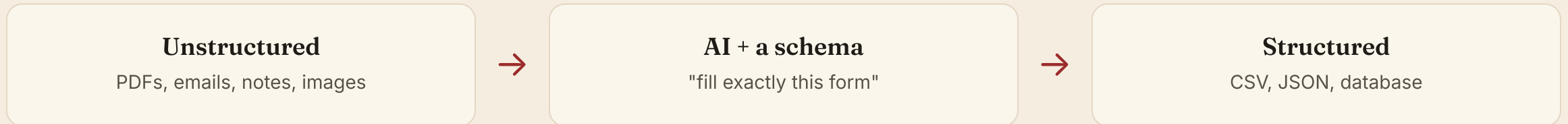
TOUCHES: Reads .pdf/.txt/.md/image files in one folder. Sends text/images to AI API. Writes one CSV (or Markdown) — only with --apply.

MUST NEVER: Modify/move/delete the source documents. Touch other folders.

DONE MEANS: One row per document, with a priority; 3 rows I spot-check match.

- The chore knowledge workers actually have: a messy pile → a sortable queue
- Same loop as yesterday's reading — now driving a real multimodal automation

The most valuable pattern in AI automation



- Once data is structured, *everything else* (sorting, sums, dashboards) is trivial
- The schema is your ticket's **DONE MEANS**, written as code

 LIVE DEMO

AUTOMATION 2 · MESSY DATA → DECISION DASHBOARD

The ticket

TASK: Turn any CSV into an interactive dashboard, plus an AI read of the notable patterns.

TRIGGER: Manual – whenever someone emails me "the numbers".

TOUCHES: Reads one CSV. Serves a local page (my machine only). Writes nothing. With --explain, sends a PROFILE (stats, not raw rows) to the AI API.

MUST NEVER: Modify the CSV. Upload the raw rows anywhere.

DONE MEANS: Browser opens; charts match 3 numbers I check; the AI read is a claim I can verify against those charts.

- The showstopper: 90 seconds from spreadsheet to something your boss thinks took a week

The line this demo teaches

Deterministic Python

- Cleans the data, draws the charts
- Exact, repeatable, trustworthy to the cell
- You *rely* on it

AI judgement (`--explain`)

- Reads the numbers, says what's interesting
- Fuzzy, fast, occasionally wrong
- You *verify* it — a lead, not gospel

Knowing which side of that line you're on is the whole skill. The AI sees a *profile*, never your raw rows — a privacy boundary you drew on purpose.

 LIVE DEMO

AUTOMATION 3 · PERSONAL INTELLIGENCE BRIEFING

The ticket

```
TASK:      Hacker News + my RSS feeds → 5-bullet AI briefing, saved as markdown.  
TRIGGER:   Every weekday, 8:00am – UNATTENDED.  
TOUCHES:   Public HN API, my RSS feeds, AI API, writes one file to ~/briefings/.  
MUST NEVER: Send/post anything. Touch any other folder. Fail silently when scheduled.  
DONE MEANS: A dated briefing file exists every weekday morning.
```

- Pick **your** sources — this is a small personal agent, not a toy fetch
- **UNATTENDED** changes everything — this ticket earns the strictest Run Gate of the course

Make it a one-word command: aliases

Mac (zsh)

Add one line to `~/.zshrc`:

```
alias briefing='uv run ~/tools/briefing.py --apply'
```

Reload: `source ~/.zshrc` — now `briefing` is a command.

Windows (PowerShell)

Add a function to your profile (`notepad $PROFILE`):

```
function briefing { uv run ~\tools\briefing.py --apply }
```

Reopen PowerShell — done.

- An alias is *just a nickname in a text file* — that's the entire trick. "Ship it" = this

T — for real this time: scheduling

Mac — launchd

- A small `.plist` file says *when*
- It calls `uv run .../briefing.py`
- `launchctl load` — done
- Template in the demo folder

Windows — Task Scheduler

- One `schtasks` command says *when*
- Same script, same uv
- Command in the demo folder
- (This is all cron ever was)

This is where every other beginner course stops short — and where automation actually begins.

Take this home: a prompt that sets up *your* machine

- `prompts/build-automation-mac.md` and `prompts/build-automation-windows.md`
- Paste into **any chatbot** (Claude, ChatGPT) — or hand to a **coding agent** (Claude Code)
- It already knows your OS's quirks: shell, profile file, scheduler, uv paths
- You fill in the ticket; it builds, aliases, and schedules the tool — **SCRIPT gates included**



YOU BUILD — 30 MINUTES

CAPSTONE · BUILD ONE REUSABLE TOOL

Your first tool

- Pick one you can describe in a sentence: **point automation 1 at your own docs** · automate **your Day-1 ticket**
· push a demo past its your-turn
- The path is the loop: ticket → `prompts/build-automation-<your-os>.md` → Run Gate → dry-run → prove
- Finish it to the **reusable-tool bar**: clear inputs · clear outputs · safe defaults · a written usage contract
- Done when you can: say what it's for in one sentence · point at every line that writes/moves/sends · show your done-means check passing

From script to a tool an *agent* can call

Take one automation you already understand. **Nothing about the Python changes.** We just write its usage contract in a form an agent reads — a one-file **skill**:

The six fields

- **Name** — what the capability is
- **When to use** — the trigger
- **Inputs** — args, files, keys
- **Command to run** — the exact `uv run ...`
- **Expected output** — what success looks like
- **Safety constraints** — the MUST NEVER lines

Why it's basically free

- A tool with clear inputs, clear outputs, safe defaults + a usage contract is **already agent-ready**
- Same Run Gate — the agent just has a **bigger reach**, so safety matters *more*
- Example: `demos/04-tool-to-skill/SKILL.md` wraps automation 1

POLL 3

Which automation maps onto *your* Monday?

1. Document inbox → structured action queue (piles of documents)
2. Messy data → decision dashboard (spreadsheets & CSVs)
3. Intelligence briefing → scheduled (fetch + summarize, unattended)
4. A reusable tool you'd want an agent to call for you

PART 4

Owning your tools

Using AI $\neq$ shipping slop

- **Slop** is unreviewed output shipped as-is. The difference isn't the AI — it's the *gate*
- You read it (I), you bounded it (ticket), you proved it (P) → **you own it**, and can say so
- "An AI wrote it" explains nothing after you've done those three things — *you* shipped it

Red → Green: tell the AI what "done" means

- Weak: *"write a script to extract invoice data"* → plausible code, unverifiable
- Strong: *"...**done means**: one CSV row per PDF, columns company/date/total; here are 2 examples"*
- Borrowed from test-driven development: **define the passing test before the code exists**
- DONE MEANS is that test in plain English — and you *run* it, every time

When a script wants to become software

- One file → five files? Others depending on it? Running it for *other people*?
- That's **software engineering** — versioning, tests, reviews — a different (learnable) game
- The skill you built here is knowing **which side of that line you're on**
- Most personal automation *never* needs to cross it. Disposable is a feature

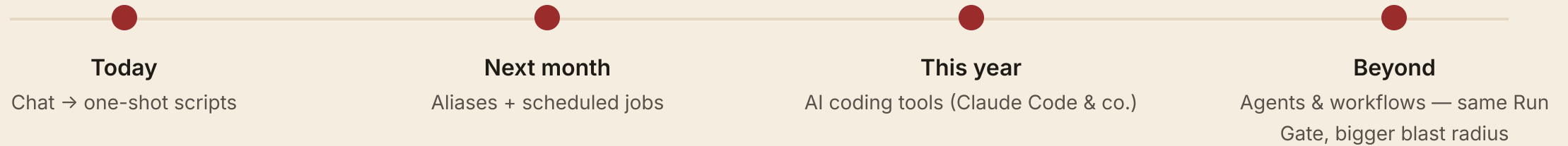
Keep learning Python — with AI, without tutorial hell

- Set a **specific goal** ("read demo 2 without help"), not "learn Python"
- Ask **targeted questions** about *your* scripts — the tutor prompt is in the repo
- When AI keeps making the same mistake: **save it** — that's your personal benchmark
- Iterate fast, write down what you learn. Loop beats lecture

Your tool hoard

- Every automation = **ticket + prompt + script**, saved together in one folder
- The hoard compounds: next tool starts from your best previous prompt
- Three solid entries today — and one you can hand to an agent. The habit is the take-home, not the files

Where this road goes



- Most people use ~5-20% of what this unlocks. You now have the ladder — **same safety habits at every rung**

The SCRIPT loop — take it with you

- **S**pot it — worth automating? already exists?
- **C**ompose the ticket — trigger · touches · must-never · done-means
- **R**equest the code — one prompt, one file, dry-run by default
- **I**nspect — Run Gate: blast radius + package check
- **P**rove — dry-run → practice → verify done
- **T**urn into a (reusable) tool — clear inputs/outputs, safe defaults, a usage contract; alias · schedule · skill

`prompts/script-loop.md` — the whole course on one page.

Resources

- The repo — lessons, demos, prompts: `github.com/EnkrateiaLucca/oreilly-python-course`
- Your-machine setup prompts: `prompts/build-automation-mac.md` · `...-windows.md`
- [uv docs](#) · [PEP 723](#) — the setup story
- [Simon Willison on one-shot Python tools](#)
- [Anthropic's AI Fluency framework](#) — the 4D vocabulary
- [Automate the Boring Stuff, 3rd ed.](#) — when you want the deep end

THANK YOU

Now go automate *one real thing*

This week. Ticket first. Dry-run first. Ship it as an alias.

lucas@automatalearninglab.com · @EnkrateiaLucca

